

Task : Get Access to the Admin Panel Using Access Control Flaws

Name : Anjelina Behera

Course : CyberSecurity

Date : 9-08-26

Objective

The objective of this assignment was to identify access control flaws in a web application, locate hard-coded credentials in client-side source code, authenticate as a regular user, and escalate privileges by manipulating session cookies to gain unauthorized access to the administrative panel.

Introduction

This assignment focused on understanding how broken access control mechanisms can allow unauthorized users to reach restricted areas of a web application. By examining the client-side source code for exposed credentials, authenticating through the login portal, and inspecting how the application handled user roles via cookies, I was able to demonstrate a privilege escalation attack using Burp Suite. The exercise highlighted why authorization checks must be enforced server-side and why sensitive credentials must never be embedded in client-delivered code.

Tools Used

- * Kali Linux - penetration testing operating system
- * Burp Suite Community Edition - HTTP proxy and request inspection
- * Web Browser - accessing the target web application
- * Browser Developer Tools - viewing page source and HTML
- * TryHackMe target machine - vulnerable web application

Prerequisites

What is Access Control?

Access control is the process of restricting what actions and resources a user can access within an application. Proper access control ensures that users can only interact with functionality and data appropriate to their role. When access control is broken or missing, attackers can bypass restrictions and access administrative or sensitive functionality.

What are Cookies?

Cookies are small pieces of data stored by the browser and sent with future HTTP requests. They are commonly used for session management, authentication, and personalization. Because cookies reside on the client side, they can be inspected and modified by the user or by proxy tools such as Burp Suite.

What is Privilege Escalation?

Privilege escalation occurs when a user gains permissions beyond those originally granted. Horizontal

privilege escalation allows access to another user's data, while vertical privilege escalation elevates a user to a higher role such as administrator. In secure applications, privilege checks must always be performed on the server before granting access to sensitive functionality.

Methodology

Step 1 - Inspecting the Client-Side Source Code

I opened the login page of the target web application and used the browser's View Page Source option to examine the underlying HTML. While reviewing the source code, I discovered a developer comment embedded near the bottom of the page that contained plaintext credentials for the exercise. The comment explicitly provided a username and password intended for initial access.

Security Observation

Embedding credentials within client-side source code is a critical security flaw because any user can inspect the page source and retrieve them. Sensitive information such as usernames, passwords, and internal notes should never be exposed in HTML, JavaScript, or other client-delivered resources.

Step 2 - Authenticating to the Application

I used the discovered credentials to log into the web application. After successful authentication, I was redirected to the employee dashboard. The page displayed a standard user interface with options such as Directory, Support, and Profile, confirming that I had logged in with regular employee privileges.

Step 3 - Intercepting HTTP Requests with Burp Suite

I configured Burp Suite as the browser's proxy and navigated through the application while intercepting traffic. I examined the HTTP request headers to understand how the application maintained the authenticated session. The intercepted request contained a cookie named `user_role` with the value set to `employee`, indicating that the application was storing authorization information directly in a client-controlled cookie.

Step 4 - Identifying the Admin Page

I examined the application structure and used Burp Suite to map out available endpoints. By reviewing the intercepted requests and responses, I identified that the application had an administrative endpoint located at `/admin.php`. Attempting to access this endpoint directly while holding an employee session resulted in a redirect or access denied message, confirming that the page was restricted.

Step 5 - Manipulating the Session Cookie

I sent the intercepted dashboard request to Burp Suite Repeater. In the request headers, I located the Cookie field which contained `user_role=employee`. I modified this value to `user_role=admin` and resent the request. The server accepted the modified cookie and returned the administrative dashboard page instead of the employee view. The application had failed to validate the user role against server-side session data and instead trusted the client-supplied cookie value.

Step 6 - Accessing the Admin Panel

With the manipulated cookie active in the browser, I navigated to `/admin.php`. The page loaded successfully

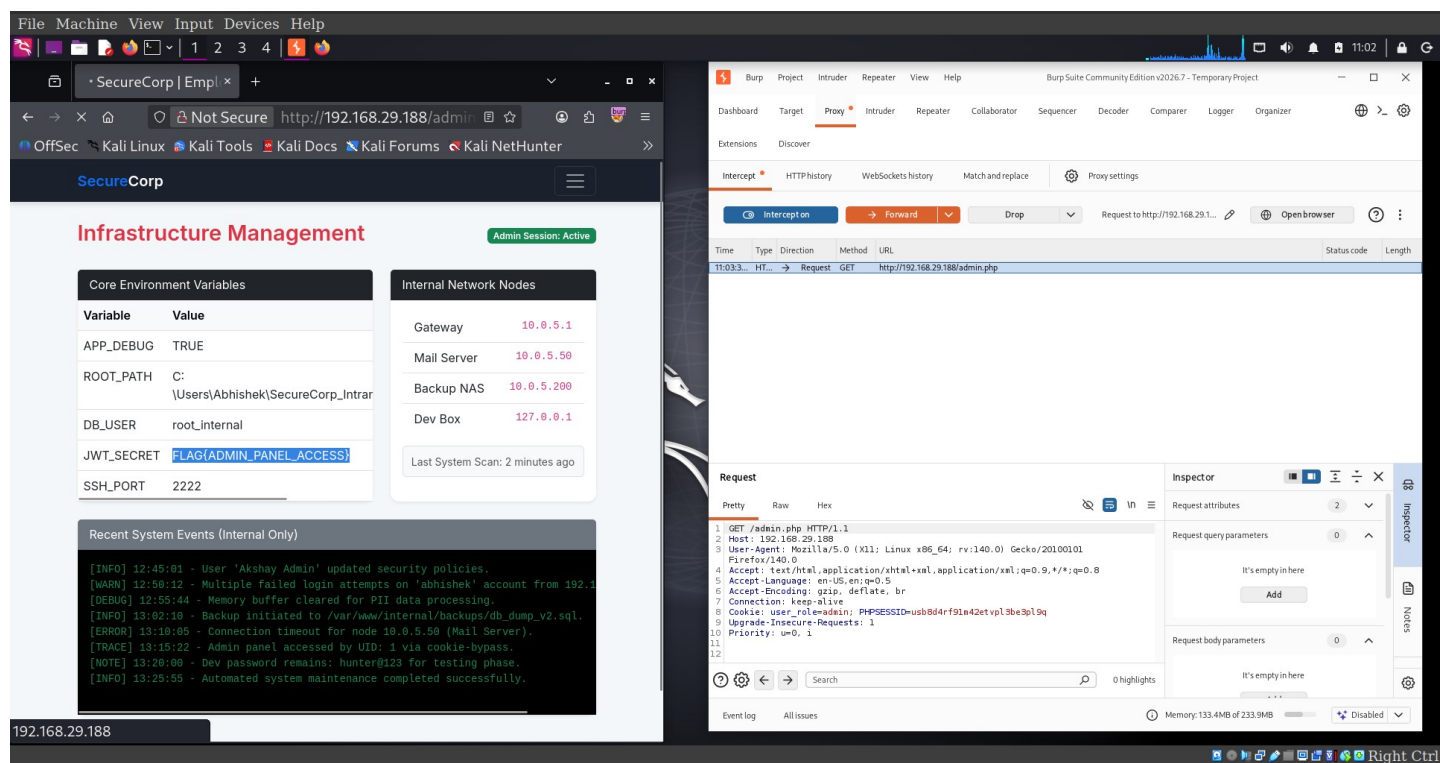
and displayed the Infrastructure Management panel, which included core environment variables, internal network nodes, and recent system events. This confirmed that the access control flaw allowed a regular user to escalate privileges and view restricted administrative functionality.

Step 7 - Locating the Flag

I examined the admin panel interface and reviewed the displayed environment variables. Among the entries, I found a variable labeled JWT_SECRET that contained the assignment flag. This demonstrated that administrative panels often expose sensitive configuration data that should never be visible to unauthorized users.

Step 8 - Screenshot Evidence

I captured screenshots of the Burp Suite Repeater showing the modified cookie request, the successful response containing the admin page content, and the browser rendering the Infrastructure Management panel with the flag visible in the environment variables table.



Security Risks Identified

1 - Exposure of Credentials in Client-Side Source Code

Hard-coded credentials in HTML comments allow any visitor to discover valid login information. Risk: Unauthorized authentication and initial foothold for further attacks.

2 - Client-Side Authorization via Cookies

The application stored the user role in a client-modifiable cookie rather than maintaining authorization state securely on the server. Risk: Any user can alter their role and bypass access controls entirely.

3 - Broken Access Control on Administrative Endpoints

The admin panel did not perform server-side validation to ensure that only authenticated administrators could access it. Once the cookie was modified, the server granted full admin access without additional verification. Risk: Complete compromise of administrative functionality, exposure of sensitive configuration data, and potential for further system-level attacks.

4 - Information Disclosure in Admin Panel

The administrative interface exposed internal environment variables including secrets and network architecture details. Risk: Attackers can leverage this information to plan secondary attacks against internal infrastructure.

Real-World Relevance

Broken access control is consistently ranked among the most critical web application security risks in the OWASP Top Ten. Real-world incidents have demonstrated that applications which rely on client-supplied role indicators or fail to enforce server-side authorization checks are vulnerable to privilege escalation and unauthorized data access. Administrative panel exposure can lead to full application compromise, data breaches, unauthorized configuration changes, and regulatory violations.

Organizations must ensure that all authorization decisions are enforced server-side, that sensitive credentials are never embedded in client-side code, and that administrative endpoints implement proper authentication and role verification before serving restricted content.

Conclusion

This assignment provided practical experience in identifying and exploiting access control flaws through client-side credential exposure and cookie manipulation. By inspecting the page source, authenticating with leaked credentials, intercepting session traffic with Burp Suite, and modifying the `user_role` cookie, I successfully escalated privileges and gained unauthorized access to the administrative panel. The exercise reinforced the importance of server-side authorization, secure session management, and the need to eliminate hard-coded secrets from client-delivered code.